



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ

«Информатика и системы управления» (ИУ)

КАФЕДРА

«Информационная безопасность» (ИУ8)

ЛАБОРАТОРНАЯ РАБОТА № 3 «Применение однослойной нейронной сети с линейной функцией активации для прогнозирования временных рядов»

по дисциплине «Введение в искусственный интеллект»

Вариант 2

Студент

ИУ8-13М
(Группа)

Бочкарев Михаил Александрович

(И. О. Фамилия)

(Подпись, дата)

Преподаватель

Басараб Михаил Алексеевич

(И. О. Фамилия)

(Подпись, дата)

Оценка: _____

Москва, 2026 г.

СОДЕРЖАНИЕ

1	Цель работы	2
2	Постановка задачи	3
3	Теоретические сведения	4
4	Программная реализация	5
5	Исследование влияния числа эпох M	6
6	Исследование влияния нормы обучения η	8
7	Исследование влияния размера окна p	9
ЗАКЛЮЧЕНИЕ		11
ПРИЛОЖЕНИЕ А Графики краткосрочного прогноза		12
ПРИЛОЖЕНИЕ Б Листинг программы		18

1 Цель работы

Исследовать применение однослойной нейронной сети с линейной функцией активации для прогнозирования временного ряда и оценить влияние числа эпох обучения, нормы обучения и размера окна на погрешность краткосрочного прогноза.

2 Постановка задачи

Для варианта 2 используется временной ряд

$$x(t) = t^4 - 2t^3 + t \quad (1)$$

на исходном интервале $[-0,5; 0,5]$. На этом интервале формируется $N = 20$ равномерно расположенных отсчётов. После обучения сети необходимо выполнить краткосрочный прогноз следующих N значений временного ряда. На каждом шаге прогнозируется один следующий отсчёт по p предыдущим реально известным значениям.

Вычислительный эксперимент проводится для параметров, приведённых в таблице 1.

Таблица 1 – Параметры вычислительного эксперимента

Параметр	Значение
N	20
a	$-0,5$
b	$0,5$
Базовый размер окна p	4
Базовая норма обучения η	1,0
Базовое число эпох M	4000
Исследуемые M	500; 1000; 2000; 3000; 4000
Исследуемые η	0,1; 0,3; 0,5; 0,7; 1,0
Исследуемые p	2; 3; 4; 5; 6; 7; 8

3 Теоретические сведения

При линейной функции активации выход нейрона совпадает с его комбинированным входом:

$$y = f(net) = net. \quad (2)$$

Для окна длины p прогнозируемое значение формируется как линейная комбинация p предыдущих отсчётов:

$$\tilde{x}_i = w_0 + \sum_{j=1}^p w_j x_{i-p+j-1}. \quad (3)$$

В реализованной программе значение w_0 фиксировано и равно нулю.

Обучение выполняется по правилу Видроу–Хоффа. Для линейной функции активации производная равна единице, поэтому изменение веса определяется выражением

$$\Delta w_j = \eta \delta x_j, \quad (4)$$

где η — норма обучения, $\delta = x_i - y_i$ — ошибка на текущем обучающем примере.

В работе исследуется краткосрочный прогноз на один шаг вперёд. Для каждого следующего отсчёта используются p предыдущих реально известных значений ряда. После вычисления прогноза в окно добавляется фактическое значение $x(t)$, поэтому ранее спрогнозированные значения не используются как входы следующих прогнозов.

Погрешность прогноза вычисляется как

$$\varepsilon = \sqrt{\sum_{i=1}^N (x(t_i) - \tilde{x}(t_i))^2}. \quad (5)$$

4 Программная реализация

Программа реализована на языке Python. Исходные $N = 20$ значений формируются непосредственно по заданной функции. Перед каждым экспериментом веса обнуляются. Для заданных p , η и M сеть обучается заново, затем строится последовательность краткосрочных прогнозов на один шаг вперёд для N следующих отсчётов и вычисляется погрешность ε .

В программе выполнены три серии экспериментов:

- изменение числа эпох M при фиксированных $p = 4$ и $\eta = 1,0$;
- изменение нормы обучения η при фиксированных $p = 4$ и $M = 4000$;
- изменение размера окна p при фиксированных $\eta = 1,0$ и $M = 4000$.

Для каждого исследуемого значения M и p программа дополнительно сохраняет отдельный график краткосрочного прогноза. Полный листинг программы приведён в приложении Б.

5 Исследование влияния числа эпох M

При исследовании числа эпох остальные параметры оставались постоянными: $p = 4$, $\eta = 1,0$. Полученные значения погрешности приведены в таблице 2.

Таблица 2 – Зависимость погрешности от числа эпох

M	Погрешность ε
500	0,052043
1000	0,048801
2000	0,043315
3000	0,038938
4000	0,035448

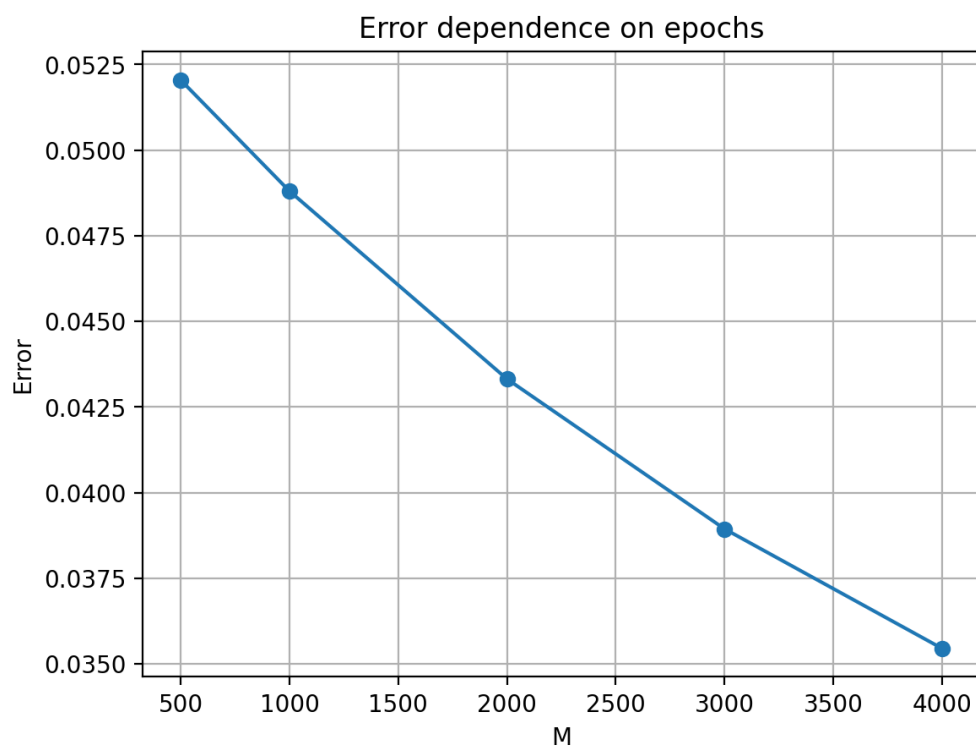


Рисунок 1 – Зависимость погрешности прогноза от числа эпох M

В исследованном диапазоне увеличение числа эпох последовательно уменьшает погрешность: с 0,052043 при $M = 500$ до 0,035448 при $M = 4000$. При выбранных p и η дополнительное обучение улучшает качество краткосрочного прогноза. Все соответствующие прогнозные графики приведены в приложении А.

6 Исследование влияния нормы обучения η

При исследовании нормы обучения использовались $p = 4$ и $M = 4000$. Результаты приведены в таблице 3.

Таблица 3 – Зависимость погрешности от нормы обучения

η	Погрешность ε
0,1	0,050614
0,3	0,045505
0,5	0,041648
0,7	0,038706
1,0	0,035448

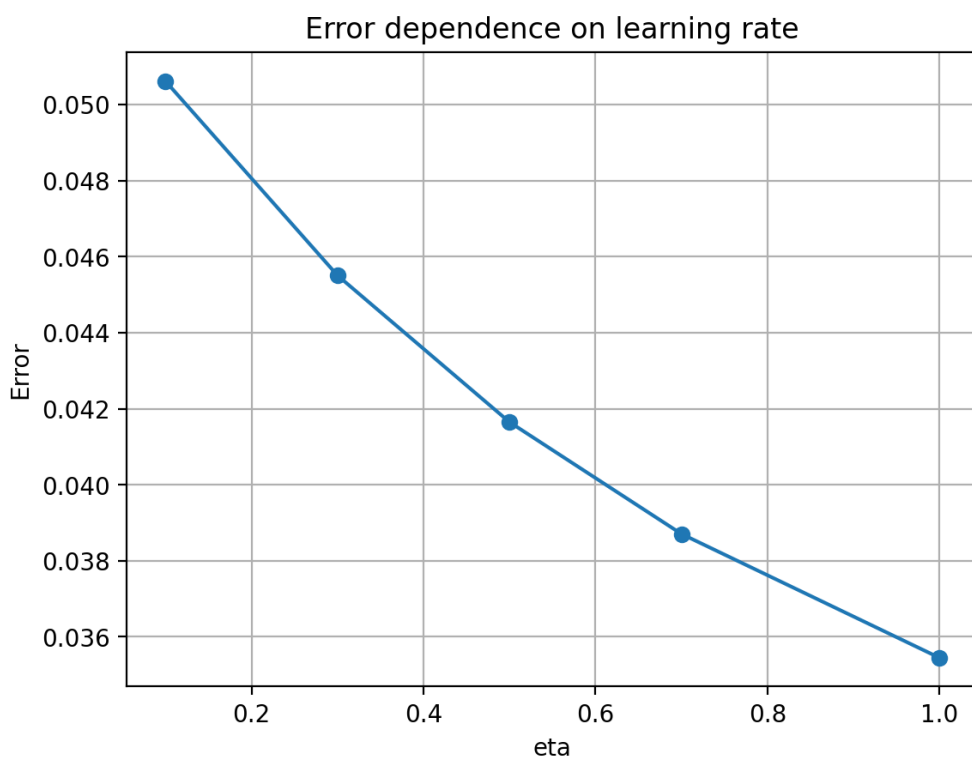


Рисунок 2 – Зависимость погрешности прогноза от нормы обучения η

В пределах исследованных значений увеличение η от 0,1 до 1,0 уменьшает погрешность с 0,050614 до 0,035448.

7 Исследование влияния размера окна p

При исследовании размера окна использовались $\eta = 1,0$ и $M = 4000$. Полученные результаты приведены в таблице 4.

Таблица 4 – Зависимость погрешности от размера окна

p	Погрешность ε
2	0,026649
3	0,035338
4	0,035448
5	0,035879
6	0,043319
7	0,054708
8	0,067427

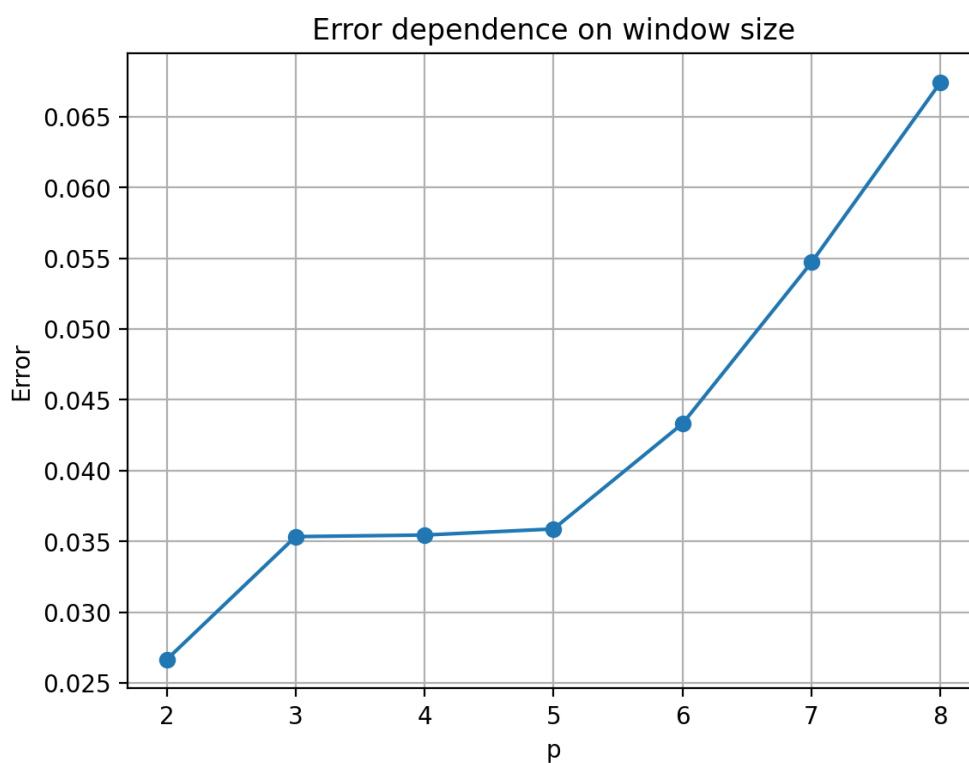


Рисунок 3 – Зависимость погрешности прогноза от размера окна p

Минимальная погрешность среди исследованных размеров окна получена при $p = 2$: $\varepsilon = 0,026649$. При увеличении размера окна погрешность в целом возрастает; для данного ряда в исследованном диапазоне короткое окно оказалось наиболее удачным. Следовательно, увеличение числа предыдущих отсчётов само по себе не гарантирует улучшения краткосрочного прогноза.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы реализована однослойная нейронная сеть с линейной функцией активации для прогнозирования временного ряда варианта 2. Выполнены вычислительные эксперименты по изменению числа эпох обучения, нормы обучения и размера окна.

В выбранном диапазоне увеличение числа эпох M и нормы обучения η сопровождалось уменьшением погрешности. Наименьшая погрешность при исследовании размера окна получена для $p = 2$. При увеличении p качество краткосрочного прогноза в данном эксперименте в целом ухудшалось. Для всех исследованных значений M и p построены отдельные графики краткосрочного прогноза на один шаг вперёд.

ПРИЛОЖЕНИЕ А

Графики краткосрочного прогноза

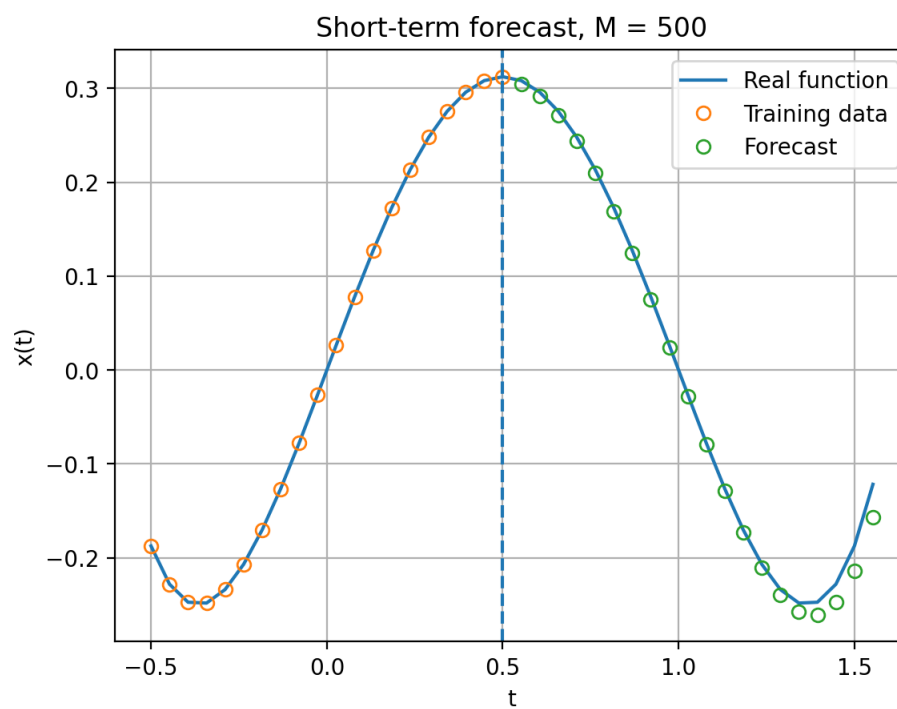


Рисунок А.1 – Прогноз при $M = 500$

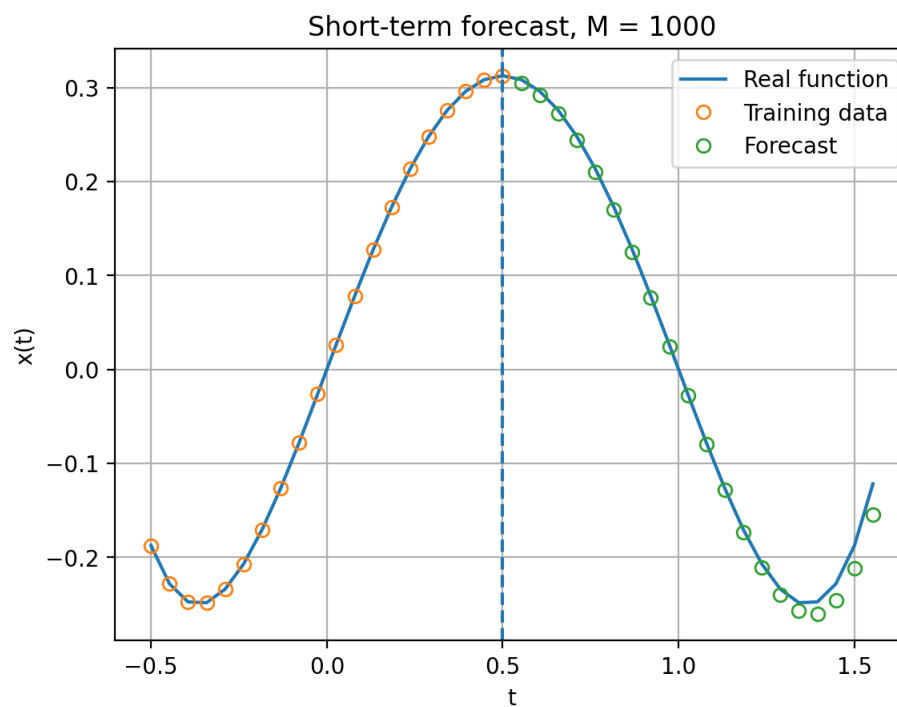


Рисунок А.2 – Прогноз при $M = 1000$

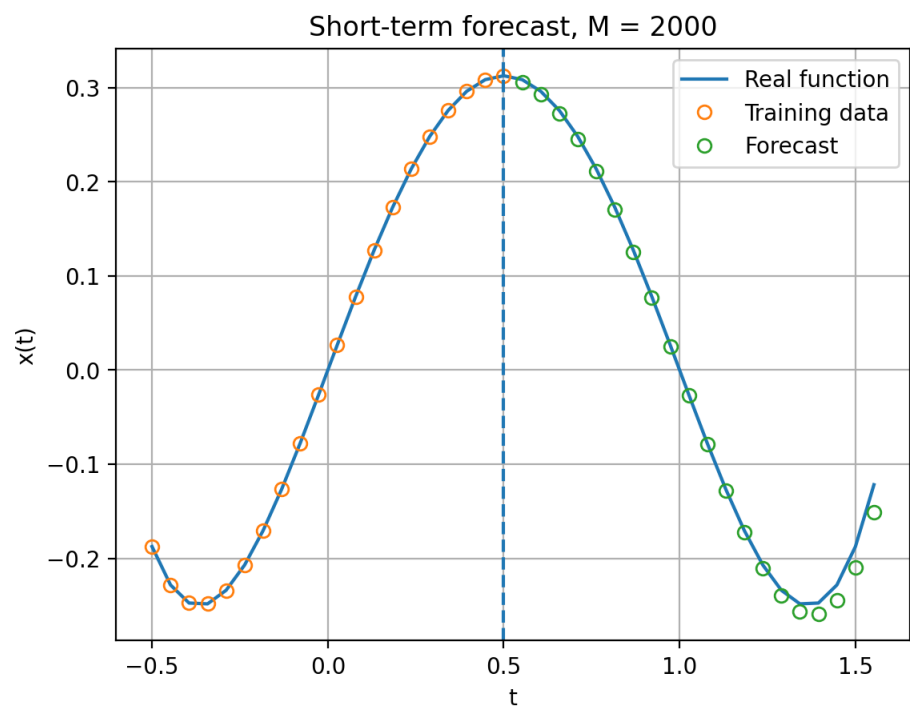


Рисунок А.3 – Прогноз при $M = 2000$

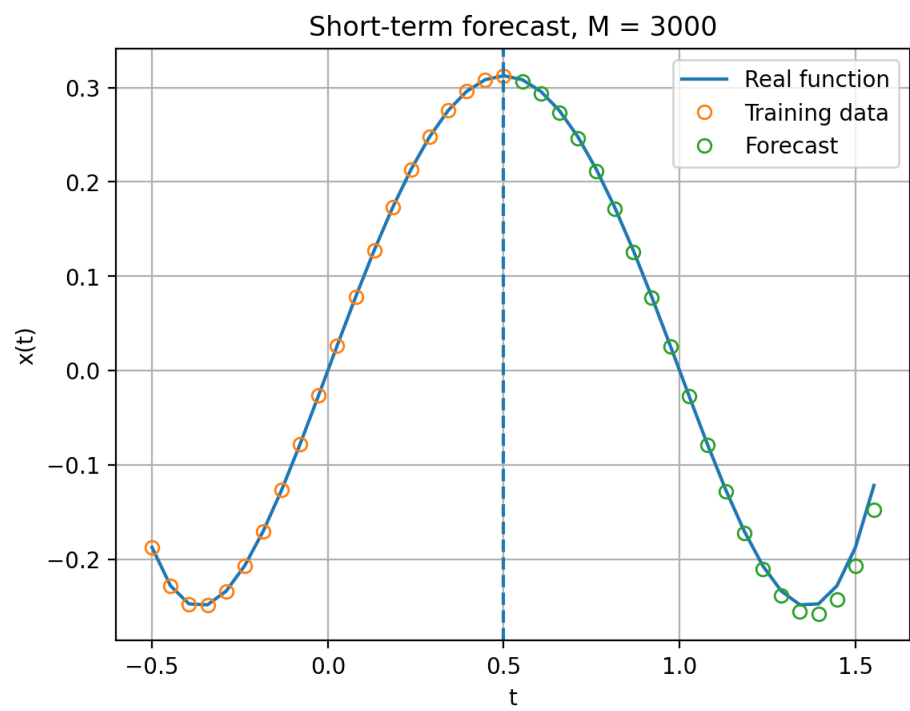


Рисунок А.4 – Прогноз при $M = 3000$

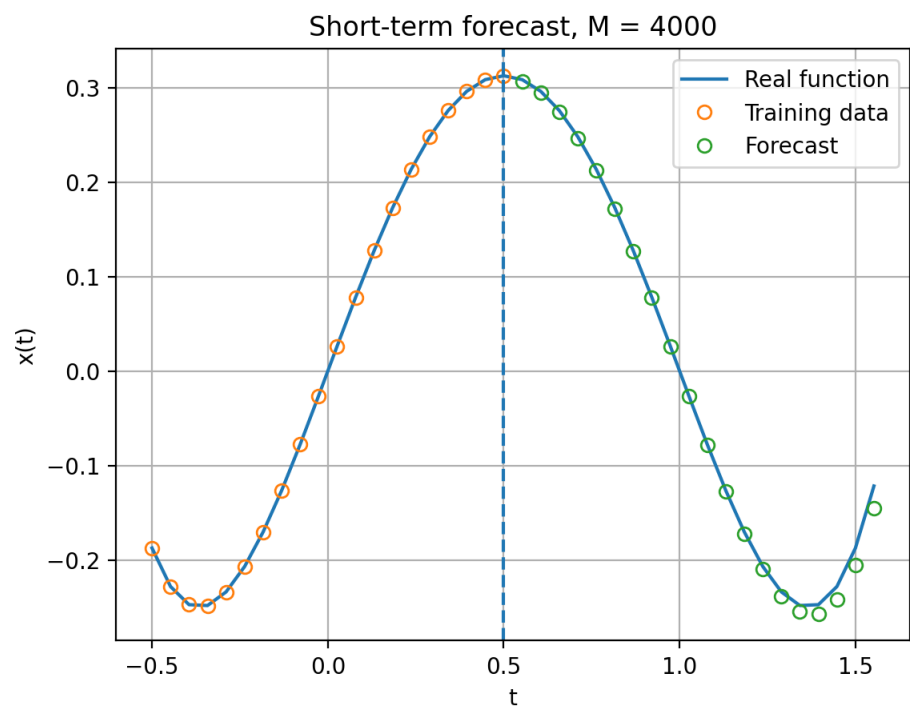


Рисунок А.5 – Прогноз при $M = 4000$

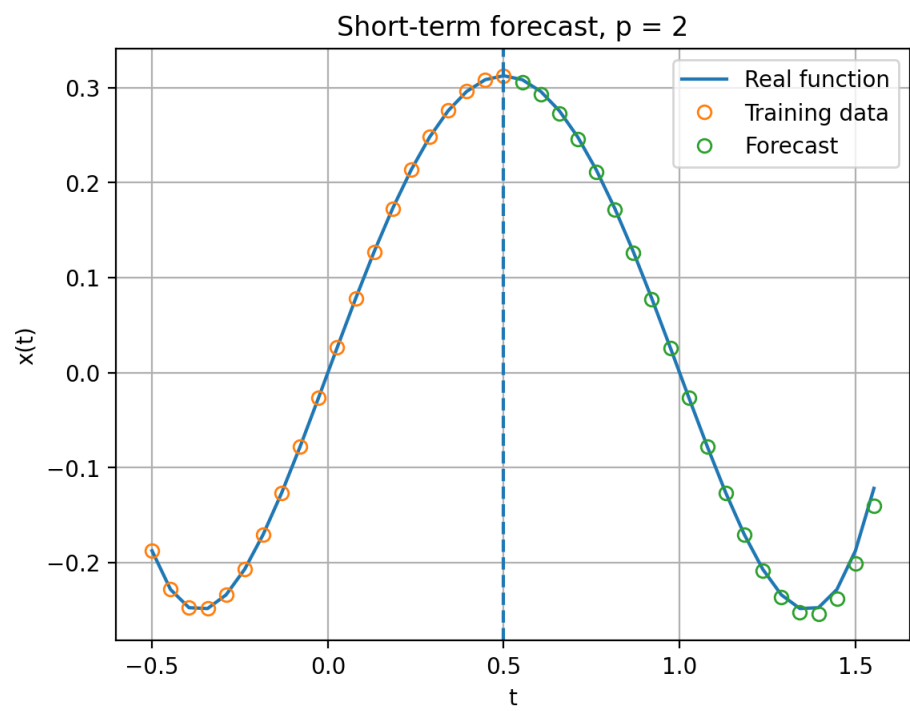


Рисунок А.6 – Прогноз при $p = 2$

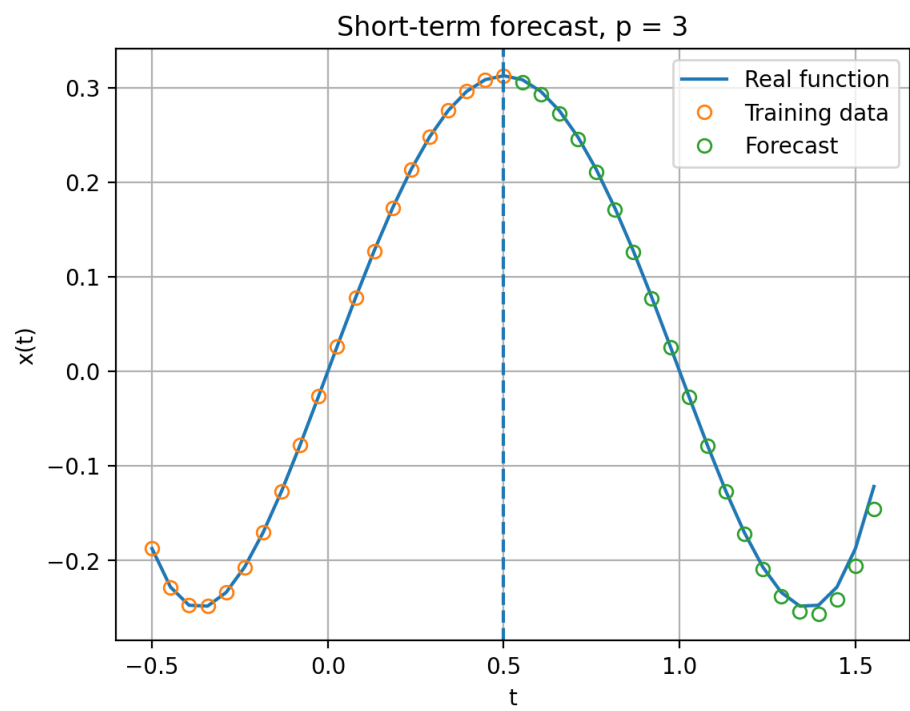


Рисунок А.7 – Прогноз при $p = 3$

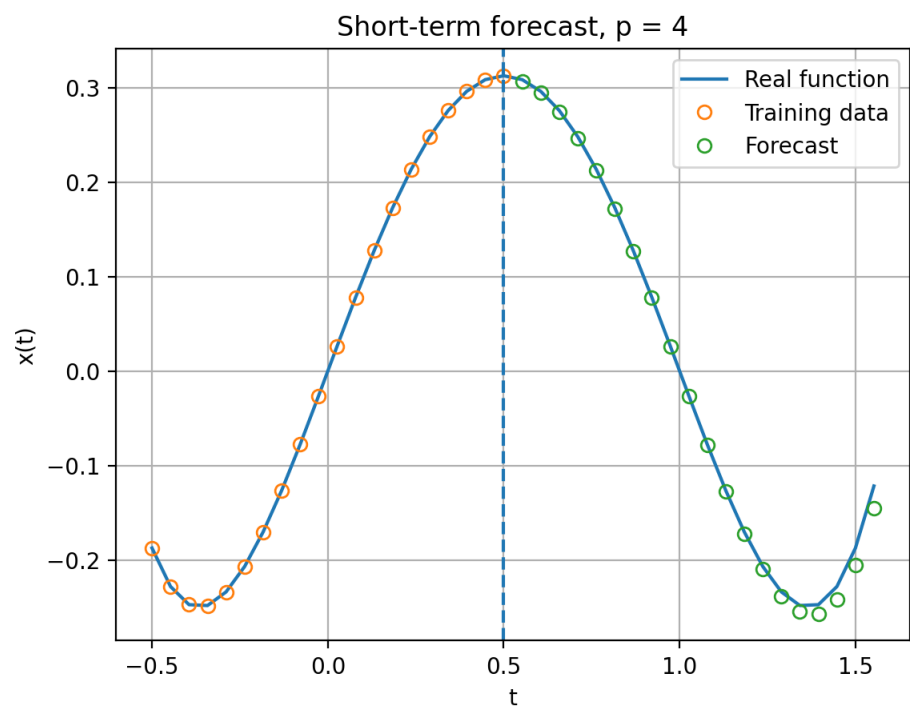


Рисунок А.8 – Прогноз при $p = 4$

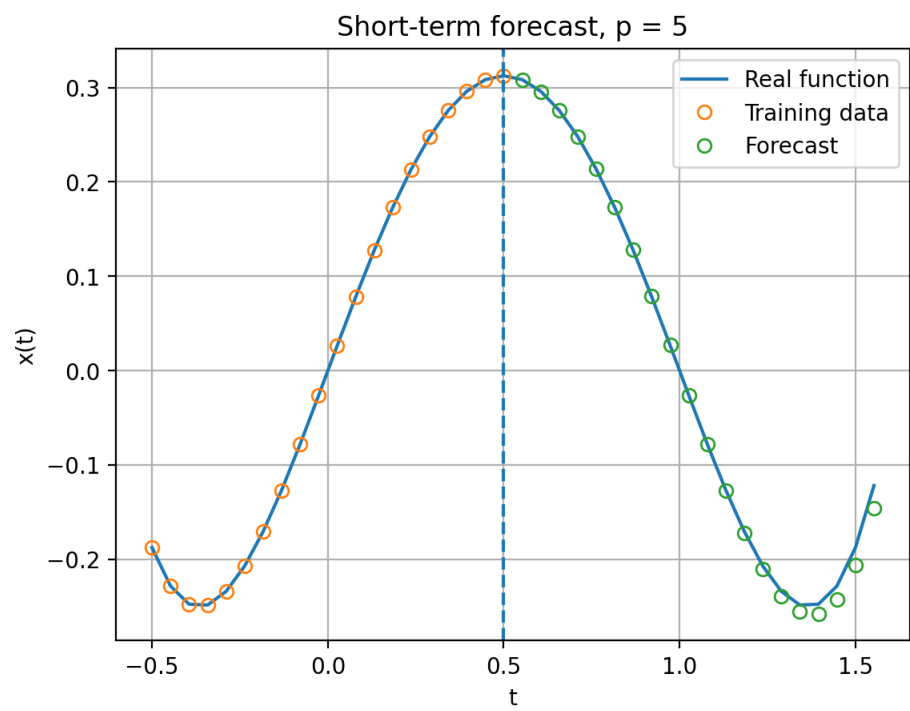


Рисунок А.9 – Прогноз при $p = 5$

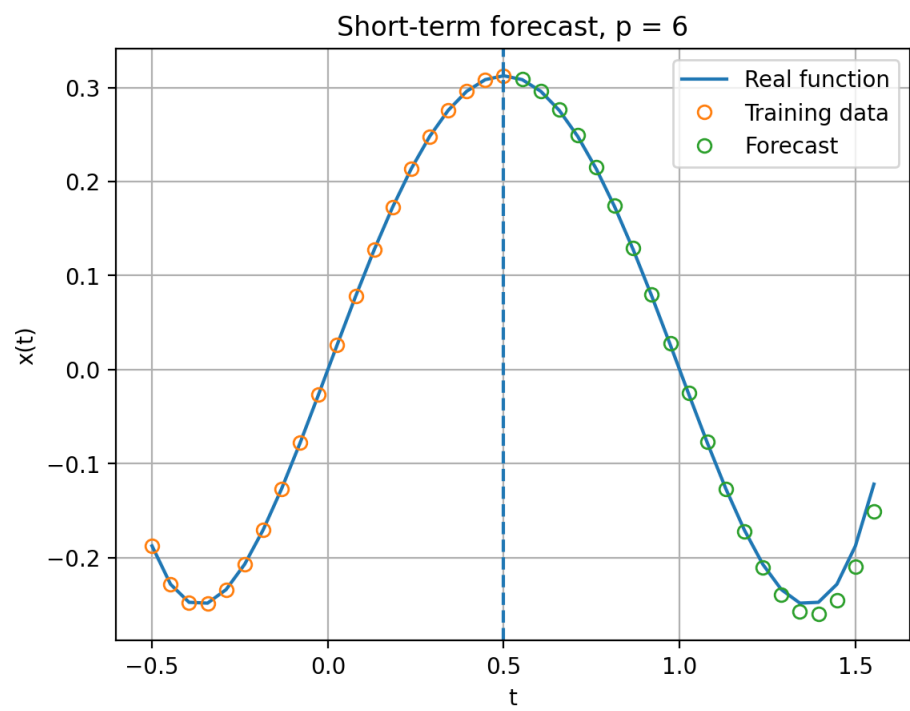


Рисунок А.10 – Прогноз при $p = 6$

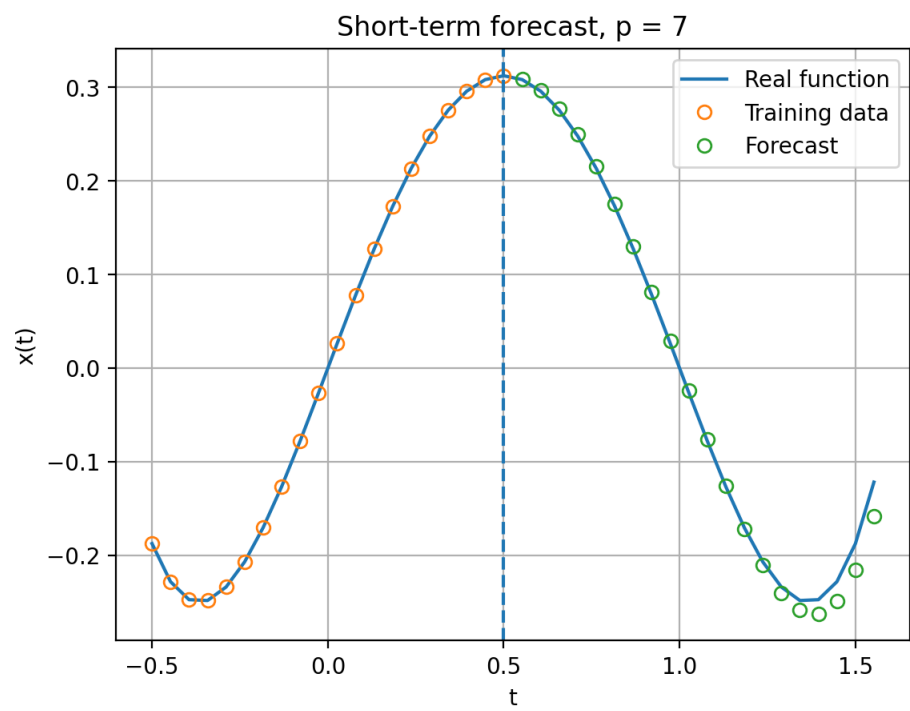


Рисунок А.11 – Прогноз при $p = 7$

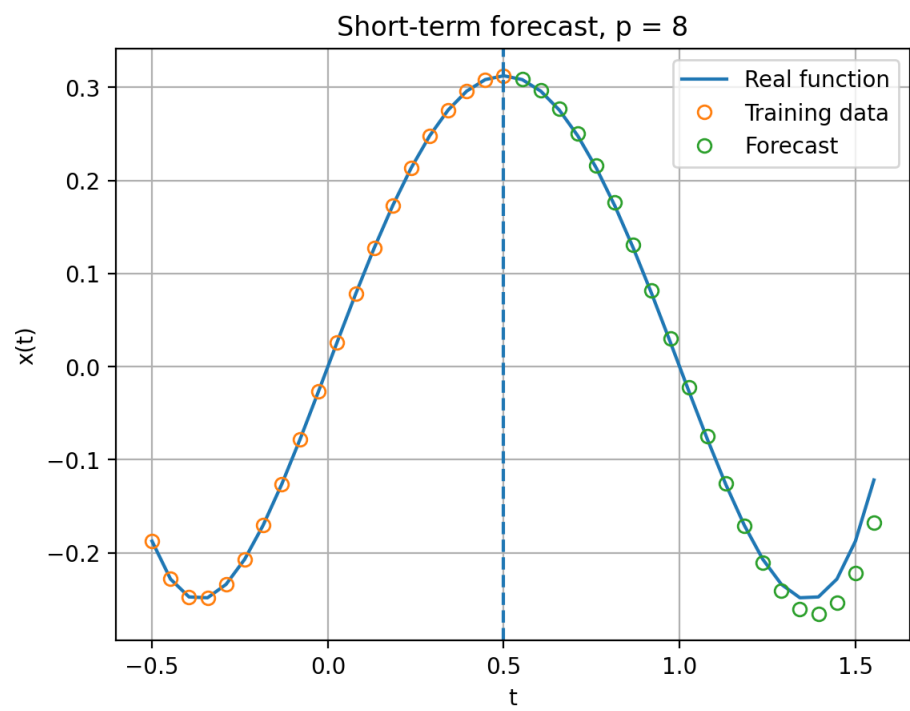


Рисунок А.12 – Прогноз при $p = 8$

ПРИЛОЖЕНИЕ Б

Листинг программы

Листинг Б.1 – lab3.py

```
import os
import matplotlib.pyplot as plt
from math import sqrt

N = 20
A = -0.5
B = 0.5

BASE_P = 4
BASE_ETA = 1.0
BASE_M = 4000

M_VALUES = [500, 1000, 2000, 3000, 4000]
ETA_VALUES = [0.1, 0.3, 0.5, 0.7, 1.0]
P_VALUES = [2, 3, 4, 5, 6, 7, 8]

os.makedirs("graphs", exist_ok=True)

def func(t):
    return t ** 4 - 2 * t ** 3 + t

def build_data():
    dt = (B - A) / (N - 1)
    train_t = []
    train_x = []
    for i in range(N):
        t = A + i * dt
        train_t.append(t)
        train_x.append(func(t))
    return train_t, train_x, dt

def train_model(train_x, p, eta, epochs):
    W = [0.0] * (p + 1)
    for epoch in range(epochs):
        for i in range(p, len(train_x)):
            net = W[0]
            for j in range(p):
```

```

        net += W[j + 1] * train_x[i - p + j]
    y = net
    error = train_x[i] - y
    for j in range(p):
        W[j + 1] += eta * error * train_x[i - p + j]
    return W

def forecast(train_x, W, p, dt):
    history = train_x.copy()
    forecast_t = []
    forecast_x = []
    for i in range(N):
        net = W[0]
        for j in range(p):
            net += W[j + 1] * history[-p + j]
        y = net
        t = B + (i + 1) * dt
        forecast_x.append(y)
        forecast_t.append(t)
        history.append(func(t))
    return forecast_t, forecast_x

def calculate_error(forecast_t, forecast_x):
    error = 0
    for i in range(len(forecast_x)):
        error += (func(forecast_t[i]) - forecast_x[i]) ** 2
    return sqrt(error)

def run_model(train_x, p, eta, epochs, dt):
    W = train_model(train_x, p, eta, epochs)
    forecast_t, forecast_x = forecast(train_x, W, p, dt)
    error = calculate_error(forecast_t, forecast_x)
    return W, forecast_t, forecast_x, error

def save_forecast(train_t, train_x, forecast_t, forecast_x, title,
filename):
    real_t = train_t + forecast_t
    real_x = []
    for t in real_t:
        real_x.append(func(t))
    plt.figure()
    plt.plot(real_t, real_x, label="Real function")

```

```

plt.plot(train_t, train_x, "o", markerfacecolor="none", label="
    Training data")
plt.plot(forecast_t, forecast_x, "o", markerfacecolor="none",
    label="Forecast")
plt.axvline(B, linestyle="--")
plt.xlabel("t")
plt.ylabel("x(t)")
plt.title(title)
plt.grid()
plt.legend()
plt.savefig("graphs/" + filename, dpi=200, bbox_inches="tight")
plt.close()

def experiment_epochs(train_t, train_x, dt):
    errors = []
    for epochs in M_VALUES:
        W, forecast_t, forecast_x, error = run_model(train_x,
            BASE_P, BASE_ETA, epochs, dt)
        errors.append(error)
        print("M =", epochs, "error =", error, "W =", W)
        save_forecast(train_t, train_x, forecast_t, forecast_x, "
            Short-term forecast, M = " + str(epochs), "forecast_M_"
            + str(epochs) + ".png")
    plt.figure()
    plt.plot(M_VALUES, errors, marker="o")
    plt.xlabel("M")
    plt.ylabel("Error")
    plt.title("Error dependence on epochs")
    plt.grid()
    plt.savefig("graphs/error_M.png", dpi=200, bbox_inches="tight")
    plt.close()

def experiment_eta(train_x, dt):
    errors = []
    for eta in ETA_VALUES:
        W, forecast_t, forecast_x, error = run_model(train_x,
            BASE_P, eta, BASE_M, dt)
        errors.append(error)
        print("eta =", eta, "error =", error, "W =", W)
    plt.figure()
    plt.plot(ETA_VALUES, errors, marker="o")
    plt.xlabel("eta")

```

```

plt.ylabel("Error")
plt.title("Error dependence on learning rate")
plt.grid()
plt.savefig("graphs/error_eta.png", dpi=200, bbox_inches="tight")
plt.close()

def experiment_window(train_t, train_x, dt):
    errors = []
    for p in P_VALUES:
        W, forecast_t, forecast_x, error = run_model(train_x, p,
            BASE_ETA, BASE_M, dt)
        errors.append(error)
        print("p =", p, "error =", error, "W =", W)
        save_forecast(train_t, train_x, forecast_t, forecast_x, "
            Short-term forecast, p = " + str(p), "forecast_p_" + str
            (p) + ".png")
    plt.figure()
    plt.plot(P_VALUES, errors, marker="o")
    plt.xlabel("p")
    plt.ylabel("Error")
    plt.title("Error dependence on window size")
    plt.grid()
    plt.savefig("graphs/error_p.png", dpi=200, bbox_inches="tight")
    plt.close()

train_t, train_x, dt = build_data()

experiment_epochs(train_t, train_x, dt)
experiment_eta(train_x, dt)
experiment_window(train_t, train_x, dt)

```